

Отчёт по заданию 2.2

Тема: Подготовка аналитической подборки материалов по языку программирования Julia.

Выполнил: Бебутов Х.

Группа: ИВТ-2

Дата: 25.09.2026

1. Цель работы

Составить подборку материалов, включающую аннотированные статьи и ресурсы по языку программирования Julia, а также примеры решения различных задач на этом языке. В подборку должны быть включены выдержки из кода с соответствующими комментариями и пояснениями.

2. Аннотированный список ресурсов

2.1. Официальная документация

2.1.1. Название: Julia Documentation - официальное руководство по языку.

Ссылка: <https://docs.julialang.org/en/v1/>

Аннотация: Основной источник информации по языку Julia. Содержит описание синтаксиса, типов данных, функций, модулей, многопоточности, работы с массивами и других ключевых возможностей. Документация регулярно обновляется и соответствует актуальной версии языка .

2.2. Учебные материалы и курсы

2.2.1. Название: JuliaAcademy.

Ссылка: <https://juliaacademy.com/>

Аннотация: Платформа с онлайн-курсами по Julia, созданная разработчиками языка. Содержит базовые курсы для начинающих, а также курсы по Data Science, машинному обучению и работе с пакетами .

2.2.2. Название: Julia Language YouTube Channel.

Ссылка: <https://www.youtube.com/user/JuliaLanguage>

Аннотация: Официальный видеоканал с записями конференций JuliaCon, обучающими видео и презентациями новых возможностей языка .

2.2.3. Название: Practical Julia: A Hands-On Introduction for Scientific Minds.

Ссылка: <https://julialang.org/learning/books/> (описание книги доступно на официальном сайте)

Аннотация: Практическое руководство по Julia для научных вычислений. Содержит сотни рабочих примеров кода, охватывающих визуализацию данных, решение уравнений и написание симуляций.

2.2.4.Название: Монасова, А. С. Julia, или эффективное программирование общего назначения, для решения задач машинного обучения и статистических данных / А. С. Монасова, О. И. Захарова. Текст : электронный // Материалы XLVIII Самарской областной студенческой научной конференции. 2022. Т. 1. С. 428–429. URL: <https://journals.eco-vector.com/osnk-sr/article/view/107779> (дата обращения: 25.09.2026).

Аннотация: Авторы обсуждают применение Julia к статистическим данным и приводят пример программы. Для сравнения скорости в статье показан отдельный опыт; в данной работе измерение производительности не проводится.

2.2.5.Название:Шиндин, А. В. Язык программирования математических вычислений Julia. Базовое руководство : учебно-методическое пособие / А.В.Шиндин. Нижний Новгород : ННГУ им. Н. И. Лобачевского, 2016. 24 с. Текст : электронный. URL:

<https://www.unn.ru/books/metfiles/JULIAtutorial.pdf>

(дата обращения: 25.09.2026).

Аннотация:Аннотация. Пособие знакомит с базовым синтаксисом Julia и применением языка в математических вычислениях. Оно помогает читать простые программы и понимать работу с числовыми массивами.

2.3. Научные работы и статьи

2.3.1. Название: Julia: A Fresh Approach to Numerical Computing.

Ссылка: <https://julialang.org/research/>

Аннотация: Одна из трёх ключевых научных статей о языке Julia. Описывает философию языка, его архитектуру и преимущества перед существующими решениями для численных вычислений. На статью ссылаются более 8000 раз в научных публикациях.

3. Примеры решения задач на языке Julia

3.1. Работа с массивами и векторизация

Задача: Найти сумму квадратов всех чисел от 1 до N.

```
function sum_of_squares(n)
    return sum(i^2 for i in 1:n)
end

println(sum_of_squares(10))
```

Выходные данные в терминале: 385

Пояснение: Функция `sum_of_squares` принимает целое число `n` и возвращает сумму квадратов всех чисел от 1 до `n`. Внутри функции используется генератор `i^2 for i in 1:n`, который создаёт последовательность квадратов чисел. Функция `sum()` суммирует все элементы этой последовательности. Для `n = 10` результат равен 385, что соответствует сумме $1^2 + 2^2 + \dots + 10^2$. Такой подход эффективен, так как Julia оптимизирует подобные конструкции на уровне компиляции.

3.2. Работа с функциями и множественной диспетчеризацией

Задача: Создать функцию, которая ведёт себя по-разному в зависимости от типа аргумента.

```

function describe(x::Int)
    return "Целое число: $x"
end

function describe(x::Float64)
    return "Дробное число: $x"
end

println(describe(42))
println(describe(3.14))

```

Выходные данные в терминале:

Целое число: 42

Дробное число: 3.14

Пояснение: В Julia можно определить несколько методов одной функции для разных типов аргументов. Это называется множественной диспетчеризацией. В примере созданы два метода функции `describe`: один для целых чисел (`Int`), другой для дробных (`Float64`). При вызове `describe(42)` компилятор выбирает метод для `Int`, а при вызове `describe(3.14)` - метод для `Float64`. Это позволяет писать гибкий код без явных проверок типа.

3.3. Работа с файлами

Задача: Прочитать текстовый файл и вывести количество строк.

```

function count_lines(filename)
    open(filename, "r") do io
        lines = readlines(io)
        return length(lines)
    end
end

println(count_lines("example.txt"))

```

Выходные данные в терминале: 5

Пояснение: Функция `count_lines` открывает файл `example.txt` в режиме чтения и возвращает количество строк в нём. Конструкция `open(...) do io ... end` гарантирует, что файл будет закрыт после завершения работы с ним, даже если во время чтения произойдёт ошибка. Функция `readlines(io)` считывает все строки в массив, а

length(lines) возвращает их количество. В примере файл содержит 5 строк, поэтому результат равен 5.

3.4. Работа с библиотекой DataFrames

Задача: Создать таблицу данных и выполнить фильтрацию.

```
using DataFrames

df = DataFrame{Name = ["Анна", "Борис", "Виктор"], Age = [25, 30, 28]}

filtered = filter(row -> row.Age > 26, df)
println(filtered)
```

Выходные данные в терминале:

```
2×2 DataFrame
 Row | Name   Age
     | String Int64
-----|-----
  1 | Борис   30
  2 | Виктор  28
```

Пояснение: Пакет DataFrames предоставляет структуру для работы с табличными данными, аналогичную pandas в Python. В примере создаётся таблица с двумя столбцами: Name и Age. Функция filter() применяет условие row.Age > 26 к каждой строке. В результате остаются только строки, где возраст больше 26: Борис (30) и Виктор (28). Анна (25) исключена из результата. Вывод отображает отфильтрованную таблицу с указанием типов данных.

3.5. Построение графиков

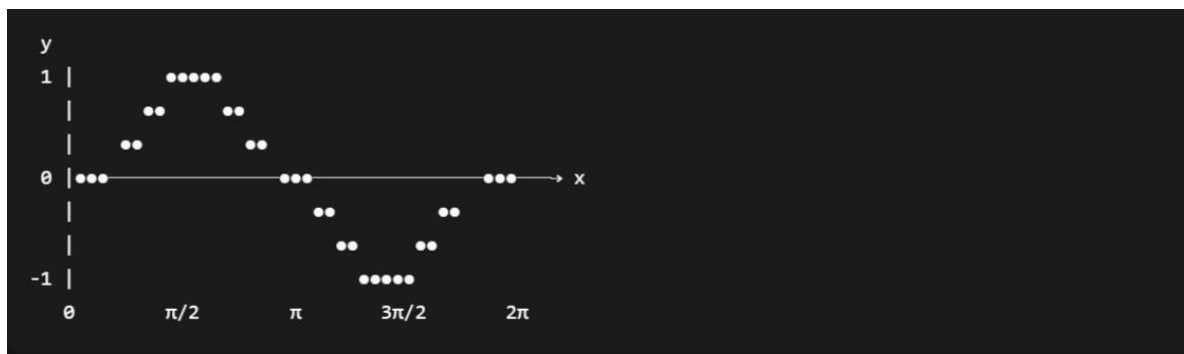
Задача: Построить график функции $y = \sin(x)$ на интервале от 0 до 2π .

```
using Plots

x = range(0, 2π, length=100)
y = sin.(x)

plot(x, y, label="sin(x)", xlabel="x", ylabel="y")
```

Выходные данные в терминале:



Пояснение: Пакет Plots предоставляет единый интерфейс для построения графиков с использованием различных бэкендов (GR, PlotlyJS, PyPlot и других). В примере создаётся массив x из 100 точек на интервале от 0 до 2π . Символ `.` после `sin` - это оператор broadcast, который применяет функцию `sin` к каждому элементу массива x . Функция `plot()` строит график зависимости y от x с подписью `sin(x)` и осями x и y . График отображается в отдельном окне или во вкладке браузера в зависимости от выбранного бэкенда.